

บทที่ 3

โปรแกรมทดสอบโดยวิธี White-box Testing และ วิธี Black-box Testing

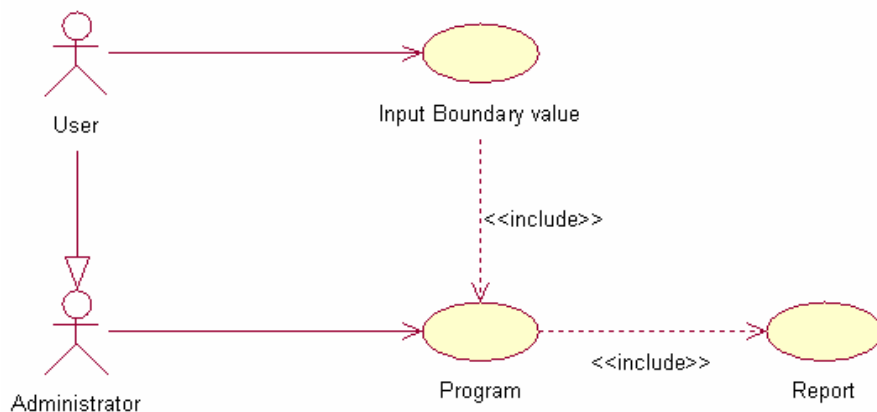
ในกระบวนการออกแบบ และสร้างกรณีทดสอบ ผู้ทดสอบจะมีหน้าที่ในการศึกษาและออกแบบกรณีทดสอบ เพื่อให้ตรงตามวัตถุประสงค์ของการทดสอบมากที่สุด ซึ่งขั้นตอนการออกแบบและสร้างกรณีทดสอบเป็นขั้นตอนที่ยุ่งยาก ใช้เวลานาน และกรณีทดสอบที่สร้างขึ้นยังมีเป็นจำนวนมากทำให้ยากในการตรวจสอบ และติดตามผลการทดสอบ ซึ่งเครื่องมือที่พัฒนาขึ้นสามารถช่วยผู้ทดสอบจัดการกับปัญหาข้างต้นได้ เป็นการช่วยลดภาระของผู้ทดสอบ

สำหรับในการทำวิจัยนี้ ผู้ทำการวิจัยได้เลือกทำการทดสอบโปรแกรมภาษาซี แต่หากว่ามีข้อจำกัดในเรื่องของเวลาในการทำวิจัย หากทำการวิเคราะห์ทางด้านโครงสร้างทั้งหมดของภาษาแล้วอาจต้องใช้เวลามาก ทางด้านผู้ทำการวิจัยจึงได้กำหนดขอบเขตของโปรแกรมภาษาซีที่จะนำมาทำการทดสอบให้แคบลง ดังนั้นโปรแกรมที่ผู้เขียนเขียนมาเพื่อทำการทดสอบจะต้องเขียนให้อยู่ในขอบเขตที่ทางผู้วิจัยได้กำหนดไว้ จึงจะสามารถนำมาตรวจสอบในชิ้นงานที่ทางผู้วิจัยได้จัดทำขึ้นได้อย่างถูกต้อง

3.1 การออกแบบ

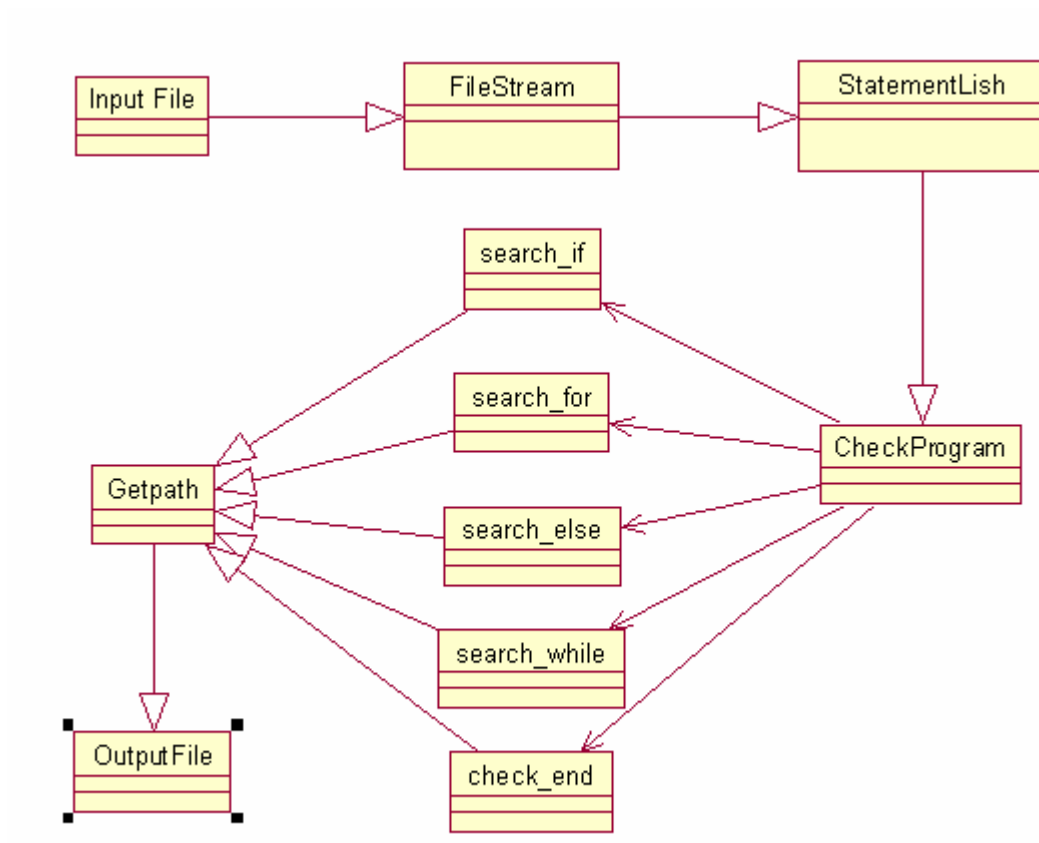
3.1.1 การวิเคราะห์และการออกแบบส่วนการหาเส้นทางการทำงานของโปรแกรม

ทำการออกแบบ Use case Diagram ให้ผู้ใช้งานทำการกำหนดช่วงของ Input ที่ต้องการทำการทดสอบให้กับโปรแกรม



รูปที่ 3.1 รูปแสดง Use Case Diagram

ทดลองสร้าง UML เพื่อจัดทำตัวโปรแกรมในส่วนของการหาเส้นทางการทำงานของโปรแกรม



รูปที่ 3.2 แสดง UML ในส่วนของการหาเส้นทางการทำงานของโปรแกรม

ตารางที่ 5.1 แสดงตารางเวลาที่ทำงานในส่วนของแต่ละทอมที่ 2

ระยะเวลา	หัวข้อการทำงาน
สัปดาห์ที่ 1	พัฒนาโปรแกรมส่วนวิเคราะห์ path ให้เสร็จ
สัปดาห์ที่ 2	ศึกษาวิธีการรันโปรแกรมในภาษา C
สัปดาห์ที่ 3	ทำโปรแกรมส่วนที่ใช้รันโปรแกรมที่ได้
สัปดาห์ที่ 4	ทำในส่วนที่รับข้อมูลจากผู้ใช้
สัปดาห์ที่ 5-6	ทำในส่วนที่สร้างกรณีทดสอบ
สัปดาห์ที่ 7-8	นำโปรแกรมแต่ละส่วนมารวมกัน
สัปดาห์ที่ 9	ทำโปรแกรมให้เป็น Application ที่ใช้งานได้
สัปดาห์ที่ 10-11	ทดสอบการใช้งานและแก้ไขข้อผิดพลาด
สัปดาห์ที่ 12	จัดทำรายงานและผลการทดลองที่ได้

3.2 การพิจารณา Condition Loop เพื่อหาเส้นทางการทำงานของโปรแกรม

เส้นทางหรือ Path ต่างๆ ในโปรแกรมนั้น จะเกิดจากการที่โปรแกรมเมอร์ได้มีการใช้ Condition Loop ต่างๆ เช่น if, for, while ในโปรแกรม ซึ่งจะทำให้เกิดการเลือกเส้นทางโดยมีค่าเงื่อนไขเป็นตัวกำหนดเส้นทางที่จะเดิน ซึ่งตัวทดสอบจะทำการรับเอา Code ของโปรแกรมมา และสร้าง Path ของโปรแกรมขึ้นมา โดยใช้กราฟ โดยแต่ละ Node จะหมายถึง 1 Statement ของโปรแกรม และลูกศรจะชี้บอก Node ต่อไป หากเป็น Loop ต่างๆ ก็จะมีลูกศรชี้ไปแต่ละ Node ตามการทำงานของคำสั่งนั้นๆ โดยจะต้องมีเงื่อนไขในการกำหนดทางเดิน Loop นั้นๆ เพื่อนำมาพิจารณาเส้นทางของโปรแกรมต่อไป

เมื่อทำการสร้างกราฟแสดง Path ของโปรแกรมเสร็จสิ้น ก็จะทำการพิจารณาแต่ละ Path ว่ามีความเป็นไปได้ไหม มี Loop ใดที่ไม่สามารถเข้าถึงได้ กล่าวคือ ไม่ว่าจะป้อนค่าใดๆ ก็ไม่สามารถเดินทางผ่าน Path นี้ได้ ซึ่งถ้าหากพบ โปรแกรมจะต้องทำการแจ้งเตือนให้โปรแกรมเมอร์ทราบ นอกจากนี้ยังทำการหา Error ของโปรแกรมจากการสร้าง Loop ที่ไม่สมบูรณ์ หรือค่าต่างๆ ที่ทำให้เกิด Error ในการเดินทางผ่าน Path นั้นๆ

ในการทดสอบข้างต้นนี้ จำเป็นต้องมีการสร้าง Test case หลายๆค่าเพื่อให้สามารถเข้าถึงได้ในทุก Path โปรแกรมจะทำการสร้าง Test case ที่ครอบคลุม และมีประสิทธิภาพ เพื่อให้ในแต่ละทางเลือก ได้ถูกทดสอบอย่างครอบคลุมที่สุด

ผลลัพธ์ที่ได้จากการทดสอบแบบ White-box testing นั้นจะบอกได้เพียงว่า โปรแกรมเมอร์สร้างโปรแกรมที่มีโครงสร้างและกระบวนการทำงานถูกต้องหรือไม่ หรือโปรแกรมสามารถทำงานได้หรือไม่ แต่ไม่สามารถบอกได้ว่าโปรแกรมทำงานได้ถูกต้องตามที่โปรแกรมเมอร์ต้องการหรือไม่ ทั้งนี้เนื่องมาจากการทดสอบแบบนี้ เน้นการทดสอบในระบบโครงสร้างโปรแกรม ไม่ได้สนใจค่าผลลัพธ์ที่ได้ว่ามีค่าเท่าใด หรือตรงตามที่ผู้ใช้ต้องการไหม หากแต่มองเพียงว่ามีผลลัพธ์เกิดขึ้นไหม และมีชนิดเป็นอะไรเท่านั้น

ดังนั้น ในการทดสอบ อาจมีการเพิ่มการทดสอบแบบ Black-box testing ลงไปเพื่อทดสอบหาความถูกต้อง และสมบูรณ์ของโปรแกรมให้มากยิ่งขึ้น

3.3 แนวความคิด และการออกแบบระบบจัดการกรณีทดสอบซอฟต์แวร์

ในการทดสอบแต่ละโครงการ ผู้ทดสอบเป็นผู้บันทึกข้อมูลรายละเอียดโครงการ และรายละเอียดของผู้ทดสอบทดสอบเข้าสู่ระบบ จากนั้นในขั้นตอนการออกแบบกรณีทดสอบ (Test cases design) ผู้ทดสอบต้องศึกษารายละเอียดและวัตถุประสงค์ของการทดสอบ เพื่อให้กรณีทดสอบที่สร้างขึ้นครอบคลุมข้อกำหนดความต้องการ (Requirement specification) ที่ระบุไว้ รายละเอียดที่ผู้ทดสอบต้องนำไปใช้ออกแบบกรณีทดสอบ มี 2 ประเภท คือ

1) ประเภทข้อมูล และลักษณะเฉพาะหรือเงื่อนไขของข้อมูลที่จะสร้างกรณีทดสอบ ตัวอย่าง เช่น ข้อมูลเข้าประเภทจำนวนจริง ซึ่งมีค่า มากกว่าศูนย์ ($x / x \in I^+ \text{ and } x > 0, I$ คือเซตของจำนวนเต็ม)

ในการทดสอบแบบเบลีกบอช ข้อมูลส่วนนี้จะนำไปใช้ในการสร้างชั้นสมมูล ข้อมูลทดสอบ และกรณีทดสอบ ตามลำดับ เครื่องมือจะรับข้อมูลเหล่านี้จากผู้ทดสอบ เพื่อนำไปสร้างกรณีทดสอบ

2) ข้อมูลรายละเอียดเพื่อใช้ประกอบการทดสอบ เช่น หมายเลขโครงการ หมายเลขรายการทดสอบ ประเภทการทดสอบ คำอธิบายการทดสอบ สิ่งแวดล้อมที่ใช้ในการทดสอบ หมายเลขกรณีทดสอบที่ต้องทำการทดสอบก่อนหน้ากรณีทดสอบนี้ (Intercase Dependencies)

ข้อมูลเหล่านี้ผู้ทดสอบทำการบันทึกเข้าสู่ระบบหลังจากที่ใส่ข้อมูลในข้อ 1 เข้าสู่เครื่องมือแล้ว เมื่อเครื่องมือทำการสร้างกรณีทดสอบเสร็จสิ้นก็จะนำข้อมูลรายละเอียดเพื่อใช้ประกอบการ

ทดสอบบันทึกลงในทุกๆกรณีทดสอบที่เกี่ยวข้องกับรายละเอียดนั้น ข้อมูลในส่วนนี้ช่วยให้ผู้ทดสอบสามารถนำกรณีทดสอบไปทำการทดสอบได้อย่างถูกต้อง

เมื่อสร้างกรณีทดสอบครบ เครื่องมือจะบันทึกกรณีทดสอบที่สร้างขึ้นลงในฐานข้อมูล และพิมพ์รายงานกรณีทดสอบ ให้ผู้ทดสอบนำไปทดสอบโปรแกรม เมื่อผู้ทดสอบทำการทดสอบเสร็จสิ้น และนำผลการทดสอบบันทึกเข้าสู่ระบบอีกครั้ง เครื่องมือจะทำการสรุป และประเมินผลการทดสอบของโครงการ และรายงานสรุปให้ผู้ทดสอบทราบความก้าวหน้าในแต่ละระดับของการทดสอบ

3.4 The syntax of Program in Backus-Naur form

ในหัวข้อนี้จะเป็นการอธิบายในส่วนของการกำหนดรูปแบบวิธีการเขียนโปรแกรมที่จะนำมาใช้ทำการทดสอบ เพื่อให้ทางผู้ที่ทำการทดสอบและทางผู้วิจัยมีความเข้าใจที่ตรงกัน

```
<Promulgate> ::= <Declare> | <Equation> | <Function>
<Declare> ::= <Type>?<Var_type><Pointer>?<Var>[ , <Var> ] *
<Type> ::= static | struct | const
<Var_type> ::= int | long | char | string | double | float
<Pointer> ::= & | *
<Equation> ::= <Var> = [ <Expression> | <Function> ]
<Function> ::= <Function_name>[ .<Function_member> ]* (<Argument>)
<Function_name> ::= <String>
<Function_member> ::= <String>
<String> ::= [A-Z a-z]+[ A-Z a-z | <Number> ]*
<Expression> ::= [ <Var> | <Number> ]+ [ <Operator><Var> | <Operator><Number> ]*
<Var> ::= <String>
<Operation> ::= + | - | * | / | %
<Number> ::= <Digit> | <Number><Digit>
<Digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<Statement> ::= <Condition>[<And_Or_exp><Condition>]*
<Condition> ::= <Expression> | <Expression><Compare_ass><Expression>
<And_Or_exp> ::= && | ||
<Compare_ass> ::= > | < | >= | <= | == | !=
```

3.5 โปรแกรมทดสอบในส่วนของ White-box Testing

ชิ้นงานของโครงการในส่วนแรกที่ทางผู้วิจัยได้จัดทำขึ้นมาก็คือในส่วน of โปรแกรมที่จะทดสอบโดยวิธีของ White-box Testing

ซึ่งผู้วิจัยได้ทำการเขียนโปรแกรมด้วยภาษา C#.Net ซึ่งในส่วนนี้จะได้แอปพลิเคชันของการทดสอบในส่วนของ White-box Testing ดังแสดงในรูปที่ 3.3



รูปที่ 3.3 แสดงแอปพลิเคชันในส่วนของการทดสอบโดยวิธี White-box Testing

ซึ่งการทำงานในส่วน of White-box Testing นั้น โปรแกรมจะทำการเปิดไฟล์ที่ทางผู้ทดสอบได้นำมาทดสอบ โดยไฟล์ที่จะทำการทดสอบจะต้องอยู่ในรูป of ไฟล์นามสกุล .C และทำการเก็บไฟล์ไว้ในโฟลเดอร์ของ Turbo C ที่เป็นตัวคอมไพล์ of ภาษาซี

3.6 โปรแกรมทดสอบในส่วนของ Black-box Testing

ชิ้นงานของโครงการในส่วนที่สองที่ทางผู้วิจัยได้จัดทำขึ้นมาก็คือในส่วน of โปรแกรมที่จะทดสอบโดยวิธีของ Black-box Testing

ซึ่งผู้วิจัยได้ทำการเขียนโปรแกรมด้วยภาษา C#.Net ซึ่งในส่วนนี้จะได้อัปพลิเคชันของการทดสอบในส่วนของ Black-box Testing ดังแสดงในรูปที่ 3.4

The screenshot shows a web-based application for Black-box Testing. It consists of three main panels. The top panel, titled 'Set Argument Range', includes input fields for 'Max Value' and 'Min Value', a 'Next' button, and labels for 'Argument Name' and 'Argument type' with a 'submit' button. The bottom-left panel, titled 'Argument Value 1', has an input field, a 'Submit' button, and labels for 'Argument Type' and 'Argument Name' with a dropdown menu set to 'Auto Assigned Vali'. The bottom-right panel, titled 'Test Result', features a 'Result' label, an input field, and 'Result' and 'Again' buttons.

รูปที่ 3.4 แสดงแอปพลิเคชันในส่วนของการทดสอบโดยวิธี Black-box Testing

การทำงานในส่วนของการทดสอบโดยวิธี Black-box Testing นั้น โปรแกรมจะทำการตรวจสอบว่า ภายในโปรแกรมที่นำมาทดสอบนั้น มีการรับค่า Argument ใดบ้างที่จะนำไปใช้ จากนั้นโปรแกรมจะทำการตรวจสอบชนิดของ Argument และจะให้ผู้ทำการทดสอบทำการกำหนดขอบเขตของ Argument แต่ละตัวเพื่อที่จะนำไปทดสอบต่อไป

หลังจากนั้นผู้ทดสอบก็จะต้องทำการกำหนดค่าของ Argument ภายในขอบเขตที่ได้กำหนดเพื่อนำไปทำการตรวจสอบผลลัพธ์ที่ได้ว่า ได้ผลลัพธ์ถูกต้องตามที่ผู้ทำการทดสอบได้ทำการเขียนโปรแกรมนั้นมาหรือไม่